

Reviewer Pack — Positivstellensatz Rank Deficiency in Max-CUT SOS Relaxation...

Entry a7db7b29ff30 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 05:45:26 UTC

Positivstellensatz Rank Deficiency in Max-CUT SOS Relaxations

Entry ID: a7db7b29ff30 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 05:45:26 UTC

1. Conjecture

Field A (mathematical branch): REAL_ALGEBRAIC_GEOMETRY **Field B** (complexity object): SUM_OF_SQUARES_HIERARCHY

Statement:

For any degree- d SOS relaxation of Max-CUT on n variables, the moment matrix M_d must satisfy $\text{rank}(M_d) \geq \Omega(n^{\{1.5\}})$ to achieve approximation ratio better than 0.878. If a relaxation achieves 0.879-approximation with $\text{rank}(M_d) = o(n^{\{1.5\}})$, then its Gram matrix violates the Positivstellensatz condition for non-negative polynomials on the hypercube.

Rationale (proposer's reasoning):

The Positivstellensatz characterizes non-negative polynomials on the hypercube as sums of squares plus terms from the feasible region. A rank deficiency in the SOS moment matrix would imply the relaxation cannot capture the necessary algebraic structure to approximate Max-CUT beyond the Goemans-Williamson threshold. This connects spectral properties of the moment matrix to algebraic certification of positivity, avoiding natural proofs by focusing on structural rather than combinatorial barriers.

Taxonomy category: SOS_DEGREE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 17b5d003e5b65181

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 16
    c = 0.5

    # Generate a random Max-CUT instance on n variables
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        G[i][i] = 0

    # Compute the degree-2 SOS relaxation
    M_2 = [[0.0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(i, n):
            if G[i][j] == 1:
                M_2[i][i] += 1
                M_2[j][j] += 1
                M_2[i][j] -= 0.5
                M_2[j][i] = M_2[i][j]

    # Compute the rank of the moment matrix
    rank_M_2 = compute_rank(M_2)

    # Check if the relaxation achieves 0.879-approximation with rank(M_d) = o(n^{1.5})
    if rank_M_2 < c * n ** 1.5:
        return {
            "metric_name": "rank(M_2)",
            "metric_value": rank_M_2,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Positivstellensatz violation"
        }
    }
```

```

# Check Positivstellensatz compliance for generated Gram matrices
if not is_positivstellensatz_compliant(M_2):
    return {
        "metric_name": "rank(M_2)",
        "metric_value": rank_M_2,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "Positivstellensatz violation"
    }

    return {
        "metric_name": "rank(M_2)",
        "metric_value": rank_M_2,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

def compute_rank(matrix):
    n = len(matrix)
    A = [row[:] for row in matrix]

    def gaussian_elimination(A, n):
        rank = 0
        for i in range(n):
            if A[i][i] == 0:
                swap_row = next((j for j in range(i+1, n) if A[j][i] != 0), None)
                if swap_row is None:
                    continue
                A[i], A[swap_row] = A[swap_row], A[i]
            for j in range(n):
                if i == j:
                    continue
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return sum(1 for row in A if any(row))

    rank = gaussian_elimination(A, n)
    return rank

def is_positivstellensatz_compliant(matrix):
    # Placeholder for Positivstellensatz compliance check
    # This is a dummy implementation and should be replaced with actual logic
    return True

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='Positivstellensatz violation' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

e': 'Positivstellensatz violation'}
TRIAL: {'metric_name': 'rank(M_2)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': F
TRIAL: {'metric_name': 'rank(M_2)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': F
TRIAL: {'metric_name': 'rank(M_2)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': F
TRIAL: {'metric_name': 'rank(M_2)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': F
TRIAL: {'metric_name': 'rank(M_2)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': F
TRIAL: {'metric_name': 'rank(M_2)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': F
TRIAL: {'metric_name': 'rank(M_2)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': F
TRIAL: {'metric_name': 'rank(M_2)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': F
TRIAL: {'metric_name': 'rank(M_2)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': F
TRIAL: {'metric_name': 'rank(M_2)', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': F

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a4619b3.py", line 117, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a4619b3.py", line 117, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before completing evaluation; counterexamples may be invalid due to incomplete data. | next: Debug seed handling in test code and re-run with stricter error checking

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	74402
2	propose	ollama_remote	qwen3:8b	0	0	70223
3	propose	ollama_remote	qwen3:8b	0	0	55739
4	preregistration	ollama_remote	qwen3:8b	0	0	27490
5	novelty	ollama_remote	qwen3:8b	0	0	26215
6	novelty	ollama_remote	qwen3:8b	0	0	21116
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13370
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8167
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8723
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12385
11	judge	ollama_remote	qwen3:8b	0	0	25479

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 343309 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/a7db7b29ff30.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a7db7b29ff30.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a7db7b29ff30.tar.gz` (if generated)